

AI Study Companion

AI-Powered Learning & Growth Workspace

Product Requirements Document

Version: 3.0 - Candidate Challenge Edition

Target: 3 - 4 Day Prototype

Role: Full Stack AI Engineer

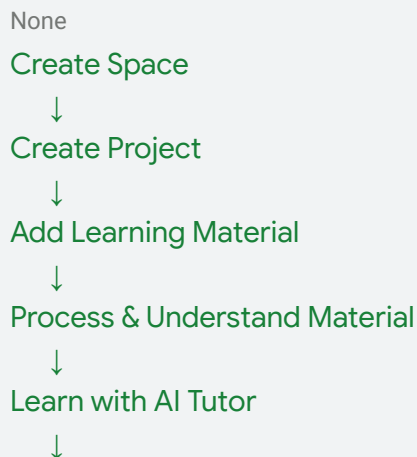
1. Product Overview

AI Study Companion is an AI-powered learning workspace designed to help users understand, practice, measure, and continuously improve a skill or area of knowledge.

The product combines learning materials, an AI Tutor, adaptive assessments, concept mastery, growth analysis, recommendations, analytics, persistent learning context, and intelligent background workflows into one connected experience.

The core idea is that learning should not be a collection of disconnected AI features. A user should be able to create a learning goal, provide relevant material, learn with an AI Tutor, test their understanding, understand their strengths and weaknesses, and receive a useful next action.

The primary learning loop is:





The product should also include an Admin Dashboard that provides visibility into users, learning activity, AI usage, AI quality, and system health.

The goal is not to build a full commercial learning platform in 3–4 days. The goal is to demonstrate strong **full-stack engineering, AI engineering, product thinking, and architectural judgment**.

2. Product Vision & Principles

The product should feel like a:

Persistent, contextual, measurable AI learning companion.

The system should continuously answer three questions:

What am I learning?

The system understands this through Spaces, Projects, goals, materials, conversations, and concepts.

How well am I learning it?

The system uses quizzes, assessments, mistakes, Tutor interactions, and mastery information as evidence.

What should I do next?

The system uses growth, learning history, weaknesses, goals, and recent activity to recommend the next useful learning action.

Several principles are fundamental.

Context First

AI interactions must respect the current Project. Information from unrelated Projects should not accidentally influence an answer.

Evidence Over Guessing

When reliable evidence is unavailable, the system should communicate uncertainty rather than confidently inventing information.

Persistent but Relevant Context

Important learning information should remain useful across sessions, but the system should avoid storing and retrieving unnecessary conversation history.

Asynchronous by Design

Long-running operations such as document processing, indexing, analytics, recommendations, and evaluations should run asynchronously where appropriate.

Observable AI

AI requests, failures, latency, retrieval, token usage, cost, and evaluation results should be sufficiently visible to allow the system to be investigated and debugged.

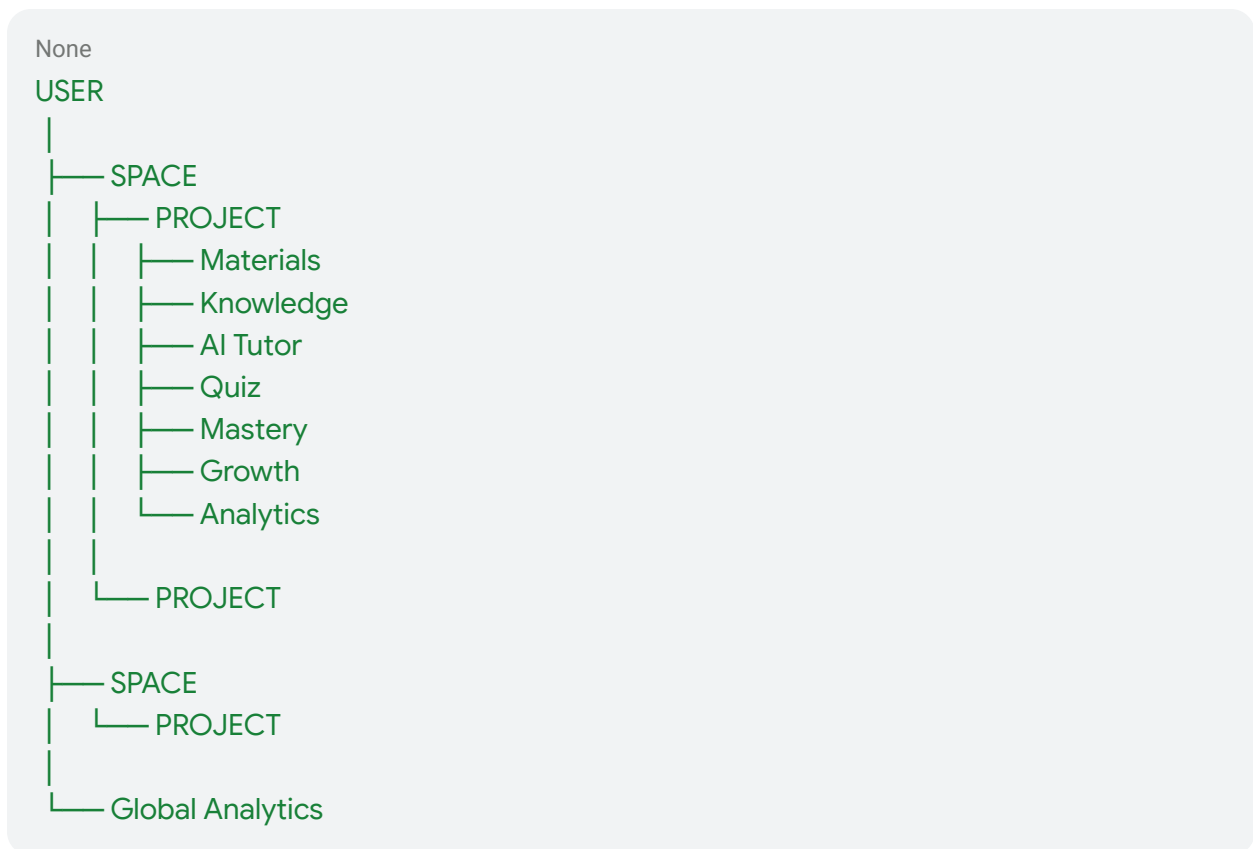
Safe AI Interaction

AI should interact with application functionality through controlled, validated, permission-aware interfaces rather than unrestricted access to internal systems.

3. Product Structure

The product is organized into **Spaces** and **Projects**.

A Space represents a broad learning area. A Project represents a focused learning journey within that Space.



Each Project should maintain its own learning context, materials, conversations, concepts, assessments, mastery, and activity.

This separation is important both for the user experience and for security/data isolation.

4. Spaces & Projects

A **Space** can represent any broad area the user wants to explore, such as a technical skill, certification, professional goal, personal interest, or creative discipline. The product should not impose a rigid category system.

A Space requires a name and description, with optional visual customization. Its dashboard should provide a high-level view of Projects, activity, progress, and areas requiring attention.

A **Project** is the core learning workspace. During creation, the user provides a name, description, and learning goal.

The Project Dashboard should summarize the current learning state, including overall progress, important concepts, recent activity, learning performance, the user's latest activity, and a recommended next step.

The user should be able to move naturally from the dashboard into:

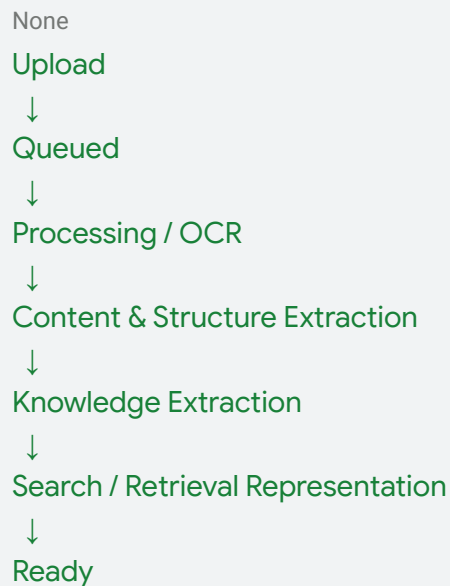
Materials → **Tutor** → **Quiz** → **Growth** → **Analytics**

5. Learning Materials & Knowledge

Users should be able to upload learning material to a Project. **PDF is the primary required format for the prototype**, although candidates may support additional formats.

Documents may contain normal text, tables, images, diagrams, or scanned pages.

After upload, processing should happen asynchronously.



The user should be able to see whether a document is queued, processing, ready, or failed.

The processing system may create chunks, concepts, metadata, relationships, page references, embeddings, or other searchable representations. The exact technology is left to the candidate.

The important requirement is that the Tutor and other AI experiences can retrieve relevant information and trace it back to its source.

Long-running work should use background processing and should support reasonable retry, failure, and duplicate-job handling.

6. AI Tutor

The AI Tutor is the primary learning experience.

The Tutor operates within the current Project and should understand the user's goal, Project materials, relevant concepts, previous conversation context, assessment history, and important learning context.

Users should be able to ask questions, ask follow-ups, request simpler explanations, explore concepts, request examples, test their understanding, or ask for revision guidance.

The Tutor should maintain useful continuity across sessions without sending the entire user history to every AI request.

The system should combine three kinds of context:

```
graph TD; A[None] --- B[Current Conversation]; B -- "+" --> C[Relevant Project Knowledge]; C -- "+" --> D[Relevant Learning Context]; D -- "↓" --> E[Context-Aware Tutor Response];
```

None
Current Conversation
+
Relevant Project Knowledge
+
Relevant Learning Context
↓
Context-Aware Tutor Response

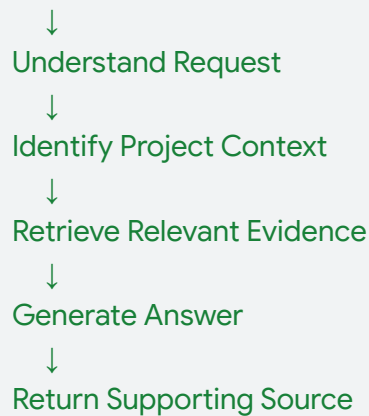
7. Grounded AI & Citations

The Tutor should prioritize the user's Project materials when answering questions.

A typical Tutor request should follow:

```
graph TD; A[None] --- B[User Question];
```

None
User Question

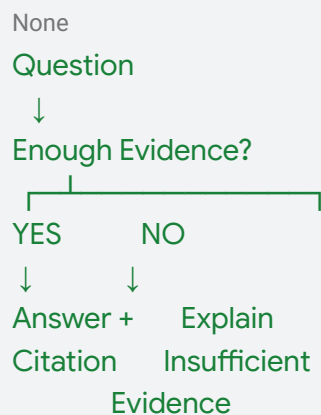


Responses should provide meaningful citations such as:

Source: Machine Learning Notes — Page 14

The user should be able to understand where the answer came from and return to the original material.

If the Project material does not contain enough evidence to answer reliably, the Tutor should not confidently fabricate an answer.



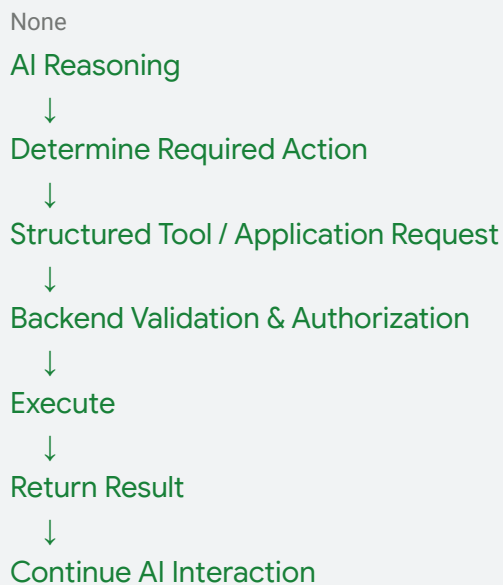
This behavior is a core evaluation requirement.

8. AI & Application Interaction

The AI layer may need to interact with application capabilities such as searching materials, retrieving progress, reading assessment history, identifying weak concepts, generating quizzes, recording learning events, updating learning state, or generating recommendations.

These capabilities should be exposed through controlled interfaces.

The expected pattern is:



The AI should not have unrestricted access to databases, internal services, or privileged application operations.

AI-generated structured data should be validated before it is persisted or used to change application state.

9. Adaptive Quiz & Assessment

The Project should provide an adaptive Quiz experience based on the user's learning material and current learning state.

The Quiz should support at least:

- Multiple-choice questions

- Open-ended questions

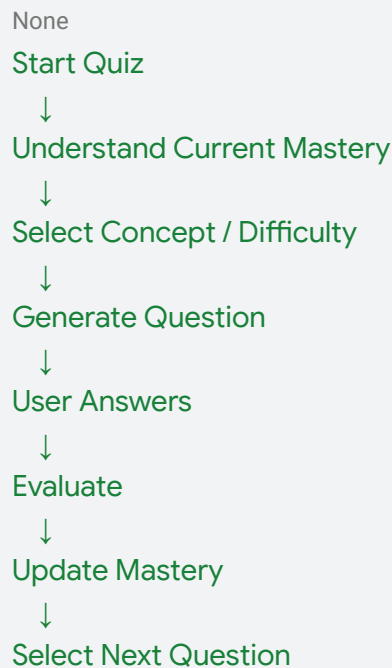
Question selection should consider concepts, mastery, previous mistakes, recent performance, difficulty, question history, and recent learning activity.

The system should not simply implement:

Wrong → Easy
Correct → Hard

Instead, it should use the available learning evidence to determine where additional practice is useful.

The core flow is:



For open-ended questions, AI should evaluate the response for factors such as understanding, accuracy, relevance, key concepts covered, missing concepts, and reasoning where appropriate.

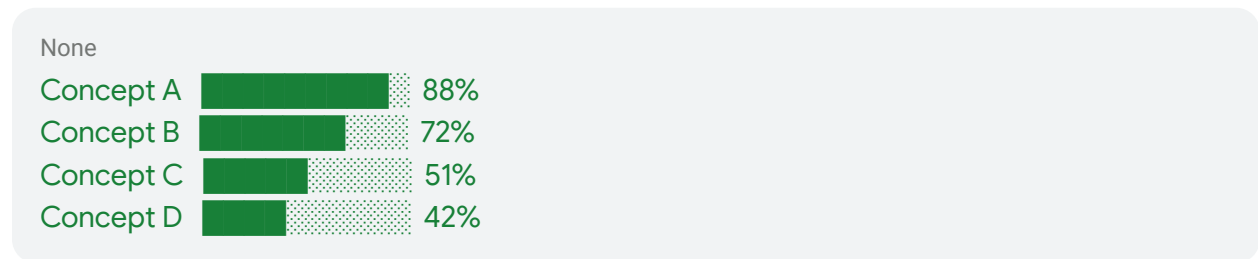
Feedback should explain what the learner understood and what is missing rather than returning only a numerical score.

Assessment results should feed into the mastery and growth systems.

10. Mastery, Growth & Recommendations

The system should maintain an estimated mastery level for important Project concepts.

For example:



Mastery is an estimate, not a claim of perfect measurement. It should evolve as new evidence becomes available through quizzes, assessments, learning activity, and other relevant interactions.

Growth Analysis should show how concepts change over time and identify areas that are:

- Improving
- Stable
- Requiring attention

The system should then convert these insights into useful recommendations.

For example:

Your understanding of Concept C has improved, but application-based questions remain difficult. Review the related material and complete another short assessment.

Recommendations may consider weaknesses, recent mistakes, goals, recent activity, assessment history, available materials, and previous recommendations.

The purpose is to answer:

What should I do next?

11. Persistent Learning Context

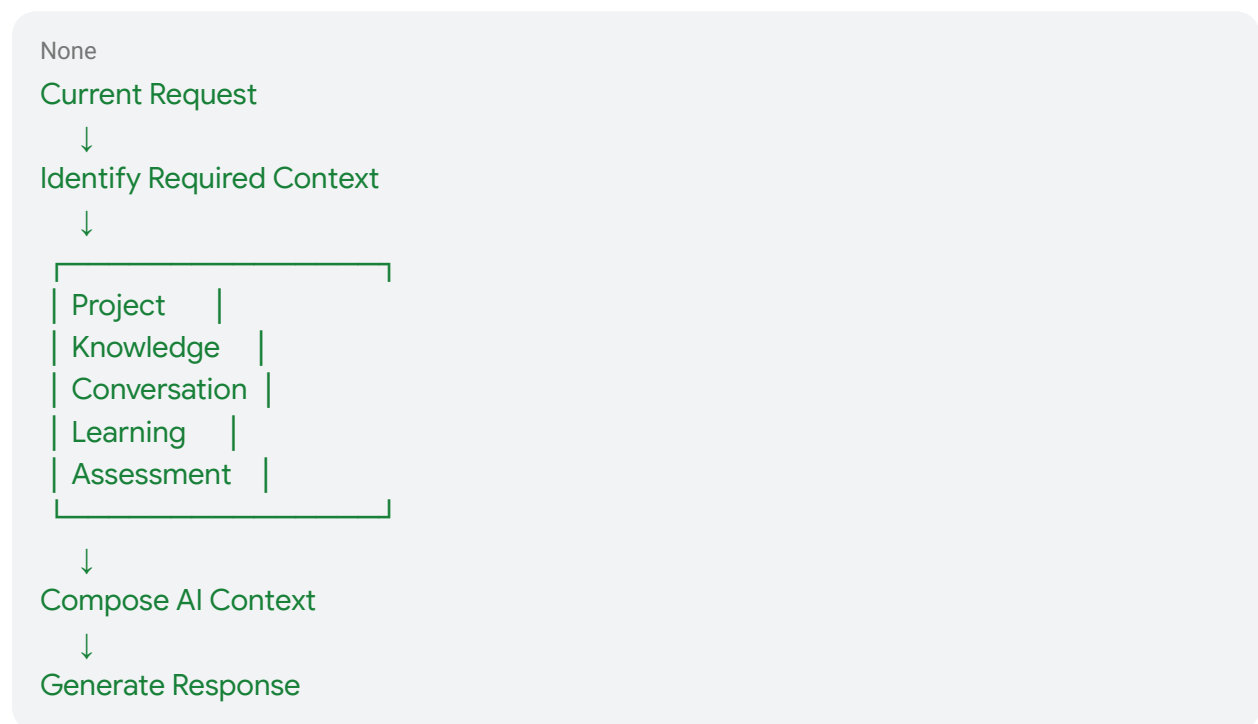
The platform should maintain a persistent representation of useful learner context.

This may include:

- Learning goals
- Relevant preferences
- Known strengths
- Known weaknesses
- Important learning history
- Significant Tutor context
- Assessment history
- Repeated mistakes

The system should prioritize relevance rather than storing everything.

When generating an AI response, the application should retrieve only context relevant to the current task.



This context is an important differentiator between a simple chatbot and a persistent learning companion.

12. Analytics & Event-Driven Learning

The platform should maintain meaningful learning events such as:

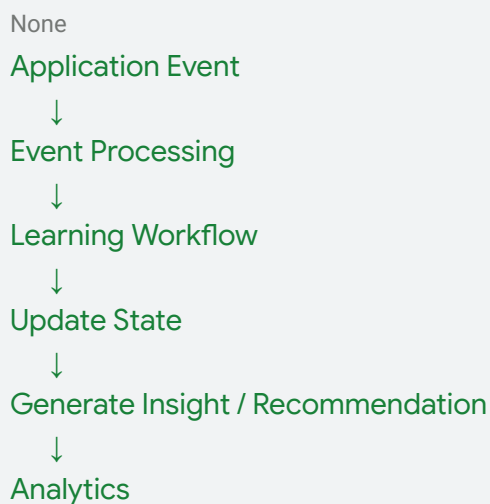
- Project creation
- Material upload and processing
- Tutor interactions
- Quiz attempts
- Questions answered
- Assessments completed
- Mastery updates
- Recommendations
- Project activity

These events should support user-facing activity, analytics, recommendations, background workflows, and administrative visibility.

Project Analytics should show learning activity, assessment performance, mastery, concept trends, and AI activity.

Global Analytics should aggregate learning activity across Projects and Spaces.

Important events may trigger downstream workflows.



For example, a completed Quiz may trigger assessment evaluation, mastery updates, weak-concept detection, and recommendation generation.

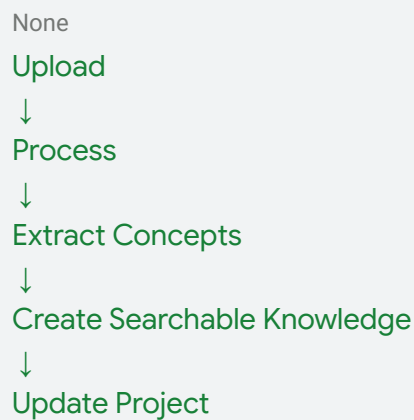
The implementation should consider retries, duplicate events, and idempotency.

13. Intelligent Background Workflows

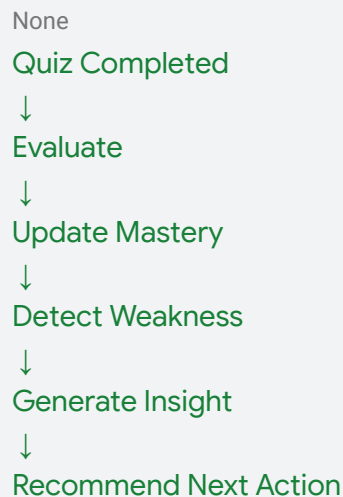
The product should use background processing for work that does not need to block the user.

Examples include:

Material workflow



Learning workflow



Repeated-mistake workflow

None

Repeated Mistake



Identify Pattern



Update Learning Context



Generate Targeted Recommendation

The system should provide reasonable job states, retries, failure handling, and recovery.

The user should not need to keep the browser open while long-running work executes.

14. AI Engineering, Observability & Evaluation

The project should demonstrate that AI is being treated as an engineering system rather than simply an API call.

Where practical, the application should abstract important AI operations such as:

- Text generation
- Structured generation
- Embeddings/retrieval
- Evaluation
- Document understanding

The exact models and providers are left to the candidate.

The system should track relevant AI usage such as model, feature, latency, token usage, estimated cost, and success/failure.

It should also be possible to investigate questions such as:

- Why was an AI response slow?
- Which model was used?
- Why did retrieval return poor context?
- Which AI workflow failed?
- How much did a request cost?

- Why did document processing fail?

AI evaluation should cover the major AI experiences.

Tutor

Accuracy, groundedness, citation correctness, and unsupported-question handling.

Retrieval

Relevance of retrieved content and source quality.

Assessment

Question quality, grading quality, structured output reliability, and adaptive behavior.

Recommendations

Relevance, actionability, and alignment with the learner state.

The candidate may use curated test cases, automated evaluation, model-based evaluation, rule-based checks, or human review.

The project should demonstrate awareness that changes to prompts, models, or retrieval can cause regressions.

15. Reliability, Security & Performance

The application should gracefully handle failures such as AI timeouts, provider failures, document-processing failures, retrieval failures, database errors, invalid AI output, rate limits, and background job failures.

Reasonable timeouts, retries, validation, logging, fallback behavior, and recovery mechanisms should be used where appropriate.

Operations that may be retried must avoid creating duplicate state.

Security is a core requirement.

Users must only access their own Spaces and Projects. Materials and retrieval must remain isolated. Background jobs must preserve ownership context, and AI application capabilities must enforce authorization.

The system should demonstrate awareness of:

- Authentication
- Authorization
- Input validation
- Data isolation
- Secure APIs
- Secure document handling
- AI-specific security

Learning materials and user messages must not automatically be treated as trusted instructions. The system should distinguish between **data, instructions, and application actions**, particularly when dealing with prompt injection or malicious content.

Performance should be appropriate for a prototype. Candidates should consider streaming Tutor responses, efficient retrieval, pagination, caching where useful, asynchronous work, efficient analytics queries, and avoiding unnecessary AI calls.

16. User & Admin Experience

User Home

The Home Dashboard should provide:

- Continue Learning
- Recent Projects
- Overall progress
- Areas requiring attention
- Recommended next action

The objective is to immediately answer:

Where was I, how am I doing, and what should I do next?

Admin Dashboard

Authorized administrators should have a platform-level view of:

- Users
- Spaces

- Projects
- Activity
- Engagement
- Learning analytics
- AI usage
- AI evaluation
- Background processing
- System health

Administrators should be able to inspect a user and understand their learning journey, including Projects, activity, assessments, progress, and AI usage.

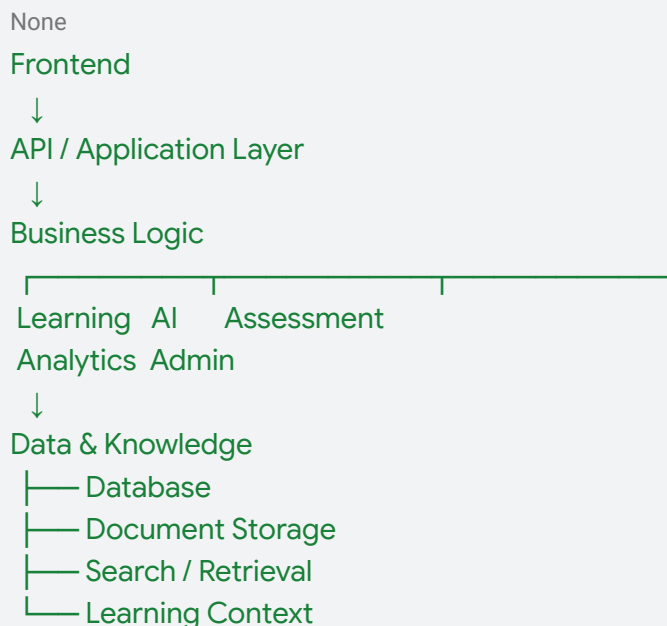
They should also be able to inspect platform-wide activity and filter it by user, Space, Project, activity type, or time period.

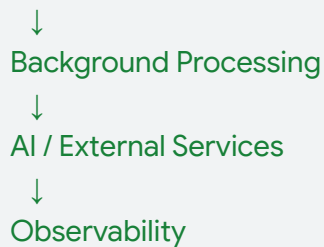
The Admin Dashboard is intended as a lightweight operational and product analytics interface rather than a replacement for dedicated infrastructure monitoring.

17. Architecture & Technology

Candidates should design an architecture with clear separation of responsibilities.

A conceptual architecture could be:





The exact architecture and technology stack are intentionally open.

Candidates may choose technologies for:

- Frontend
- Backend
- Database
- Authentication
- Document processing
- Retrieval
- AI
- Background processing
- Caching
- Observability
- Deployment

Technology choices should be justified based on requirements, reliability, development speed, maintainability, cost, and engineering judgment.

The backend should expose clean interfaces for major capabilities and demonstrate validation, authorization, error handling, and sensible separation of business logic.

The database should represent relationships between users, Spaces, Projects, materials, conversations, concepts, assessments, mastery, recommendations, activity, and AI usage.

18. Testing, Deployment & Prototype Scope

The prototype should contain meaningful tests rather than attempting exhaustive coverage.

Important testing areas include:

Backend: authentication, authorization, Project isolation, validation, and core business logic.

AI: grounded responses, unsupported questions, structured outputs, Tutor behavior, and assessment evaluation.

Learning: mastery updates, adaptive question selection, and recommendations.

Background processing: successful jobs, retries, and failure handling.

The application must be deployed to a publicly accessible URL and should demonstrate a working frontend, backend, database, authentication, AI functionality, document processing, and background processing where implemented.

Environment configuration and secrets must be separated from source code. API keys, credentials, and secrets must never be committed to the repository.

Must Have

The prototype must demonstrate:

- Authentication
- Spaces and Projects
- PDF materials
- Background document processing
- AI Tutor
- Grounded answers with citations
- Unsupported-question handling
- Adaptive Quiz
- Open-ended assessment
- Concept mastery
- Growth Analysis
- Recommendations
- Project and global analytics
- Activity tracking

- Admin Dashboard
- Persistent relevant learning context
- Project-level data isolation
- Structured AI interaction
- Basic AI observability and evaluation
- Error handling
- Testing
- Deployment
- Public repository
- Architecture documentation

Should Have

If time permits:

- Streaming Tutor
- Rich document understanding
- Improved analytics
- Persistent Tutor continuity
- Background learning insights
- Caching
- AI tracing
- Provider abstraction
- Automated regression evaluation
- Improved workflow retry handling

Nice to Have

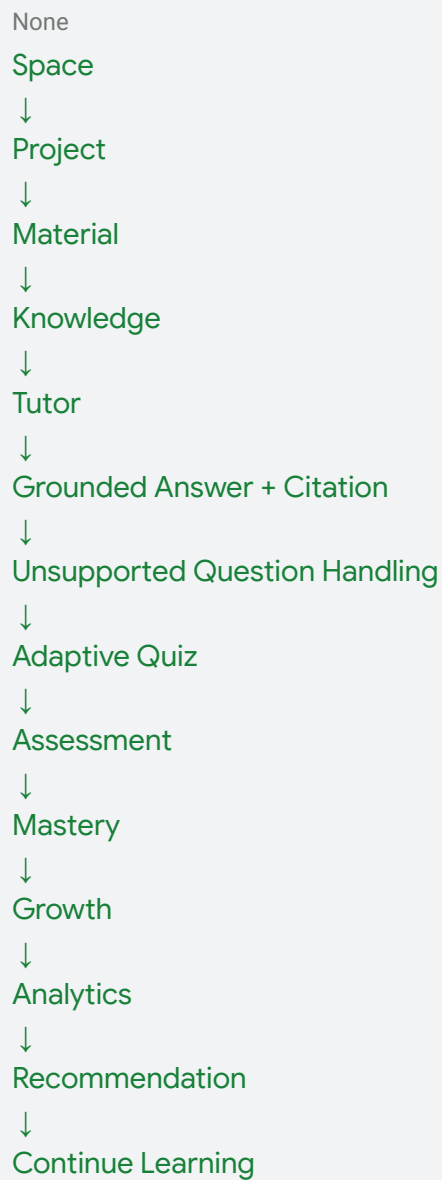
Candidates may add creative features such as:

- Voice learning
- Flashcards
- Spaced repetition
- Learning plans
- Concept maps
- Simulations
- Personalized schedules
- Multi-modal learning
- Notifications
- Collaboration
- Advanced analytics

These features should not compromise the core learning loop.

19. Success Criteria & Engineering Judgment

The primary success criterion is that a user can complete the complete learning loop without losing context:



An administrator should simultaneously be able to inspect users, Projects, learning activity, analytics, AI usage, AI evaluation, and system health.

The project should demonstrate understanding of:

- Full-stack architecture
- AI/LLM integration
- Retrieval
- Persistent context
- Structured AI outputs
- AI/application interaction
- Background processing
- Event-driven workflows
- Security
- Data isolation
- Observability
- AI evaluation
- Testing
- Performance
- Deployment

Candidates should document important engineering decisions, including what they selected, why they selected it, what they simplified, and what they would improve with additional time.

The evaluation should focus on **engineering reasoning and implementation quality**, not technology popularity.

20. Final Submission Requirements

Each candidate must submit:

1. Working Application

A deployed AI Study Companion demonstrating the core learning loop.

2. Demo Video

A short video showing:

None

Create Space



Create Project



Upload Material



Process Material



Ask Tutor



Grounded Answer + Citation



Unsupported Question



Adaptive Quiz



Open-Ended Assessment



Mastery / Growth



Analytics



Recommendation



Admin Dashboard

3. Public GitHub Repository

The repository should contain the source code, README, setup instructions, configuration examples, architecture documentation, testing instructions, and deployment information.

4. Architecture Documentation

Provide an architecture diagram and explain the major architectural decisions.

5. AI Usage Documentation

Clearly distinguish:

AI used to build the product, such as coding assistants, debugging tools, design tools, and development agents.

AI used by the final product, such as Tutor, quiz generation, assessment, recommendations, document understanding, and evaluation models.

6. Development Prompts

Provide the actual prompts materially used with AI development tools, organized where practical by architecture, frontend, backend, database, AI, debugging, testing, and documentation.

7. Evaluation Approach

Explain how Tutor quality, retrieval, assessments, recommendations, or other AI behavior was evaluated.

8. Known Limitations

Document important limitations involving AI, retrieval, documents, scaling, security, cost, UI, or background processing.

9. Future Improvements

Optionally describe what you would build next with additional development time.

21. Creativity & Differentiation

The requirements above define the **baseline experience**, not the limit of the product.

Candidates are encouraged to think beyond the explicit requirements and introduce valuable capabilities, intelligent workflows, personalization, innovative assessment methods, useful visualizations, or other AI-powered experiences.

A candidate should not be rewarded simply for implementing the largest number of features.

Instead, differentiation should come from:

- Good product decisions
- Thoughtful AI usage
- Strong learning experiences

- Intelligent automation
- Engineering quality
- Creativity
- Reliability
- User usefulness

If you identify an important problem that is not explicitly described in this PRD and solve it thoughtfully, that initiative should be considered positively.

22. Final Challenge Statement

Build an **AI Study Companion that feels less like a chatbot and more like a real learning partner.**

The system should understand what the user is learning, use their learning materials as evidence, remember relevant learning context, explain concepts, evaluate understanding, identify weaknesses, track mastery, recommend what to do next, and continuously adapt the learning experience.

At the same time, the implementation should demonstrate that you can build a modern full-stack AI system with appropriate architecture, asynchronous processing, reliable AI interactions, controlled application capabilities, data isolation, security, observability, evaluation, testing, and deployment.

The technologies, frameworks, models, and implementation strategy are intentionally open.

Show us how you would build it.

And most importantly:

Don't just build what is written. Build what you believe the product should become.